



From Errors to Execution: A Hands-On Guide to CUDA ML Setup on WSL2 or Linux



Momin Aman

Follow

8 min read · Jun 1, 2025



16



Documented, tested, and co-written by [Ahmed Affraz](#) & [Momin Aman](#)

Getting your machine ready for GPU-accelerated machine learning on Linux sounds straightforward, until you're knee-deep in mismatched library versions, obscure error messages, and conflicting installation guides.

From Errors to Execution: A Hands-On Guide to CUDA ML Setup on WSL 2



```
>>> PhysicalDevice(name="physical_
device:GPU:0', device_type='GPU')
```



This article is part guide and part dev log. On April 24, 2025, we configured and tested a complete ML environment using CUDA 12.6, cuDNN 9.6, PyTorch 2.7, and TensorFlow 2.18 on Ubuntu 24.04 via WSL2. Every step was documented and validated that day, so what you're reading isn't just theory, it's a working setup. This guide is the result of that experience: a clear, tested, and fully functional setup that goes beyond surface-level instructions.

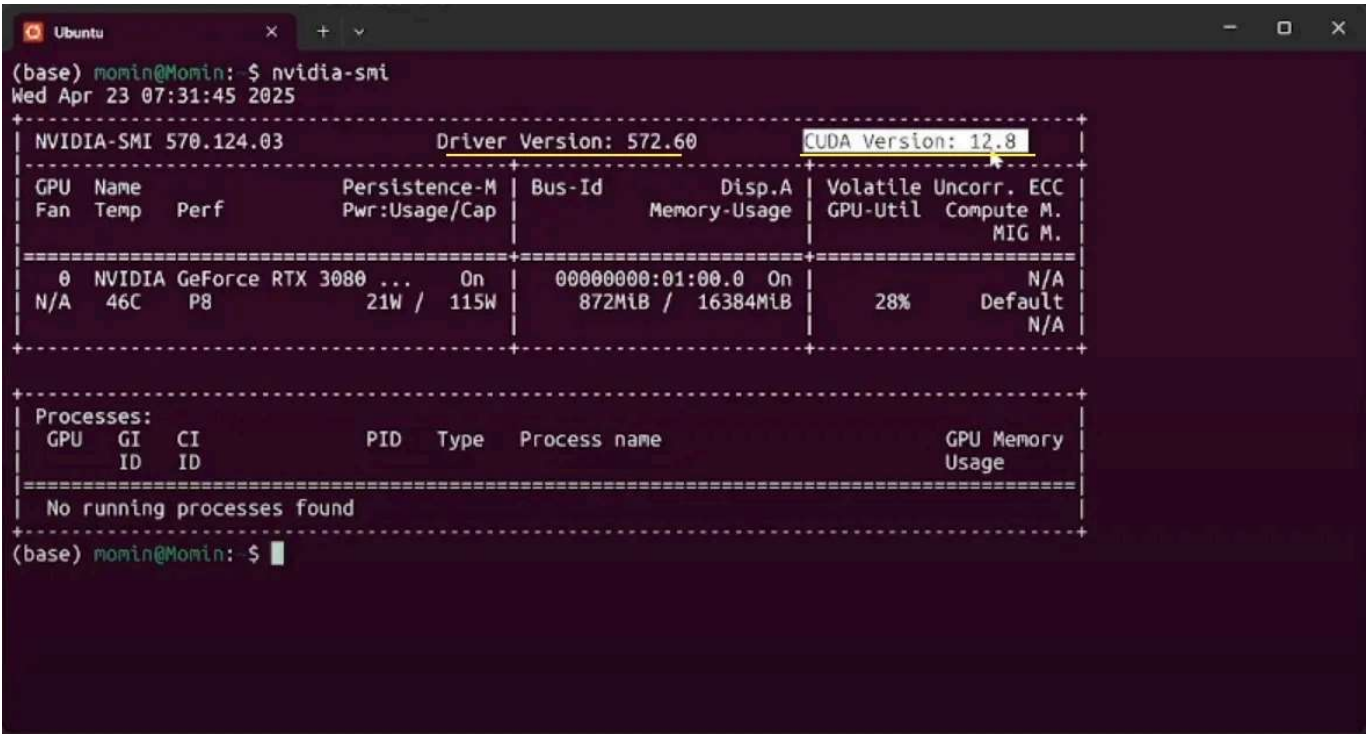
Although a of this is documented for Ubuntu 24.04 on WSL2, you can use all of the steps on native, bare-metal installation of Ubuntu as well.

Step 1: Confirming the Driver and Choosing the Right CUDA SDK

Before installing anything, we started with a basic but crucial question: **which versions are even compatible with our system?**

Everything in the CUDA ecosystem begins with your **NVIDIA driver version**. The driver determines which versions of the CUDA SDK you can install. So our first step was to confirm exactly what driver was installed.

```
nvidia-smi
```



WSL2 Driver Behavior

What’s worth noting, especially if you’re new to WSL, is that WSL2 shares the NVIDIA driver installed on your Windows host system. You don’t need to install a separate GPU driver inside Ubuntu (unlike native Linux). This simplifies the setup a bit, but it also means that your CUDA SDK must be compatible with your host system’s driver version.

Based on this driver version and NVIDIA’s support matrix, we determined that **CUDA 12.6** was the best choice, as newer versions would risk

compatibility issues with cuDNN and the frameworks we planned to use.

*We chose **CUDA 12.6** because it was the highest confirmed compatible version for our driver and it had stable support in both PyTorch 2.7 and TensorFlow 2.18.*

Step 2: Downloading and Installing CUDA 12.6.3 (WSL2)

Once we confirmed that **CUDA 12.6** was the best compatible version for our driver, the next step was to install the SDK.

We navigated to the [CUDA 12.6.3 archive page](#) and selected the appropriate settings for our environment

Note: It's important to use the WSL-specific SDK to avoid missing dependencies or driver mismatches. The installation instructions are slightly different from native Ubuntu setups. But same instructions could be used for native Ubuntu.

We opted for the **runfile-based installation** instead of a package manager (like apt) for greater control. While package managers are convenient, they often include additional dependencies or automatic driver installations, which we didn't need because **WSL uses the Windows GPU driver directly**.

Go to the following link to download the CUDA SDK Toolkit:

Website Link

https://developer.nvidia.com/cuda-12-6-3-download-archive?target_os=Linux&target

CUDA Toolkit 12.6 Update 3 Downloads

Select Target Platform

Click on the green buttons that describe your target platform. Only supported platforms will be shown. By downloading and using the software, you agree to fully comply with the terms and conditions of the [CUDA EULA](#).

Operating System	Linux	Windows						
Architecture	x86_64	arm64-sbsa	aarch64-jetson					
Distribution	Amazon-Linux	Azure-Linux	Debian	Fedora	KylinOS	OpenSUSE	RHEL	Rocky
	SLES	Ubuntu	WSL-Ubuntu					
Version	2.0							
Installer Type	deb (local)	deb (network)	runfile (local)					

Download Installer for Linux WSL-Ubuntu 2.0 x86_64

The base installer is available for download below.

> CUDA Toolkit Installer

Installation Instructions:

```
$ wget https://developer.download.nvidia.com/compute/cuda/12.6.3/local_installers/cuda_12.6.3_560.35.05_linux.run
$ sudo sh cuda_12.6.3_560.35.05_linux.run
```

Additional installation options are detailed [here](#).

Select these options to get the proper link for downloading CUDA Toolkit

```
# Download CUDA 12.6.3 SDK for WSL2
```

```
wget https://developer.download.nvidia.com/compute/cuda/12.6.3/local_installers/
```

```
# Run the installer (skip driver installation if prompted)
```

```
sudo sh cuda_12.6.3_560.35.05_linux.run
```

```
Ubuntu x + -
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

(base) momin@momin: $ g++ --version
g++ (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

(base) momin@momin: $ wget https://developer.download.nvidia.com/compute/cuda/12.6.3/local_installers/cuda_12.6.
3_560.35.05_linux.run
--2025-04-23 07:39:17-- https://developer.download.nvidia.com/compute/cuda/12.6.3/local_installers/cuda_12.6.3_
560.35.05_linux.run
Resolving developer.download.nvidia.com (developer.download.nvidia.com)... 23.46.17.66, 23.46.17.43
Connecting to developer.download.nvidia.com (developer.download.nvidia.com)|23.46.17.66|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4446722669 (4.1G) [application/octet-stream]
Saving to: 'cuda_12.6.3_560.35.05_linux.run'

cuda_12.6.3_560.35.05_linux 100%[=====] 4.14G 81.3MB/s in 67s
2025-04-23 07:40:24 (63.4 MB/s) - 'cuda_12.6.3_560.35.05_linux.run' saved [4446722669/4446722669]

(base) momin@momin: $ ls
Miniconda3-latest-Linux-x86_64.sh  cuda_12.6.3_560.35.05_linux.run  miniconda3  source
(base) momin@momin: $ sudo sh cuda_12.6.3_560.35.05_linux.run
[sudo] password for momin:
█
```

Once the installation was completed, we added the following lines to our `~/.bashrc` file to ensure the system recognizes the CUDA binaries and libraries:

Open `.bashrc` in a text editor:

```
nano ~/.bashrc
```

Then add these two lines to the end of the file:

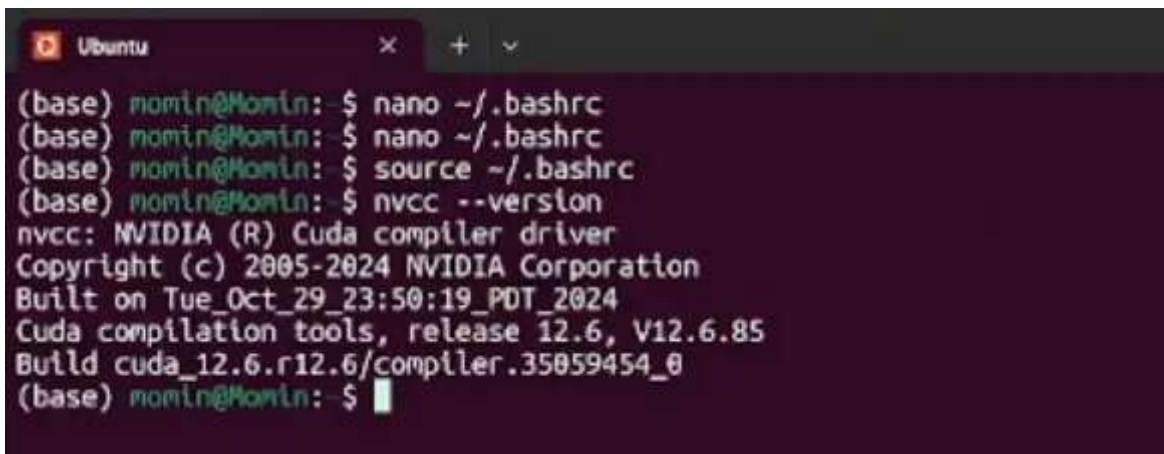
```
export PATH=/usr/local/cuda-12.6/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda-12.6/lib64:$LD_LIBRARY_PATH
```


Save and exit the file (Ctrl + O, Enter, then Ctrl + X), and apply the changes:

```
source ~/.bashrc
```

Once we had updated `.bashrc` and reloaded our shell, we verified that the CUDA toolkit was properly installed by checking the version of `nvcc`, the CUDA compiler:

```
nvcc --version
```

A terminal window titled 'Ubuntu' with standard window controls (close, maximize, zoom). The terminal shows a series of commands and their outputs. The user enters 'nano ~/.bashrc' twice, then 'source ~/.bashrc', and finally 'nvcc --version'. The output of 'nvcc --version' is displayed in white text on a dark purple background. The output includes: 'nvcc: NVIDIA (R) Cuda compiler driver', 'Copyright (c) 2005-2024 NVIDIA Corporation', 'Built on Tue_Oct_29_23:50:19_PDT_2024', 'Cuda compilation tools, release 12.6, V12.6.85', and 'Build cuda_12.6.r12.6/compiler.35059454_0'. The prompt '(base) noming@Momin:~\$' is visible at the bottom.

```
(base) noming@Momin:~$ nano ~/.bashrc
(base) noming@Momin:~$ nano ~/.bashrc
(base) noming@Momin:~$ source ~/.bashrc
(base) noming@Momin:~$ nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2024 NVIDIA Corporation
Built on Tue_Oct_29_23:50:19_PDT_2024
Cuda compilation tools, release 12.6, V12.6.85
Build cuda_12.6.r12.6/compiler.35059454_0
(base) noming@Momin:~$
```

Step 3: Installing cuDNN 9.6 for Deep Learning

After setting up the CUDA toolkit, the next critical component was **cuDNN** (CUDA Deep Neural Network library). cuDNN provides optimized primitives for deep learning workloads and is essential for frameworks like PyTorch and TensorFlow to use the GPU efficiently.

We chose cuDNN 9.6 as it is fully compatible with CUDA 12.6 (per [NVIDIA's support matrix](#)).

Support Matrix						
GPU, CUDA Toolkit, and CUDA Driver Requirements						
The following sections highlight the compatibility of NVIDIA cuDNN versions with the various supported NVIDIA CUDA Toolkit, CUDA driver, and NVIDIA hardware versions.						
Supported NVIDIA Hardware and CUDA Version						
cuDNN Package [1]	CUDA Toolkit Version	Supports static linking? [2]	NVIDIA Driver Version for Linux	NVIDIA Driver Version for Windows	CUDA Compute Capability	Supported NVIDIA Hardware
cuDNN 9.6.0 for CUDA 12.x	› 12.6	Yes	≥525.60.13	≥527.41	› 9.0 [3]	› NVIDIA Hopper [3]
	› 12.5				› 8.9 [3]	
	› 12.4				› 8.6	› NVIDIA Ada Lovelace architecture [3]
	› 12.3				› 8.0	
	› 12.2				› 7.5	
	› 12.1				› 7.0	› NVIDIA Ampere architecture
	› 12.0				› 6.1	
					› 6.0	› NVIDIA Turing
					› 5.0	› NVIDIA Volta
						› NVIDIA Pascal
						› NVIDIA Maxwell

CUDA Toolkit v12.6 is compatible with cuDNN v9.6.0

Download and Setup

We followed instructions from the [cuDNN 9.6.0 archive for Ubuntu 24.04 \(local deb installer\)](#).

Before installation, we added the required dependencies to compile cuDNN samples:

```
sudo apt-get install libfreeimage3 libfreeimage-dev
```

Then, we downloaded and installed cuDNN from following link:

```
https://developer.nvidia.com/cudnn-9-6-0-download-archive?target\_os=Linux&target
```



cuDNN 9.6.0 Downloads

Select Target Platform

Click on the green buttons that describe your target platform. Only supported platforms will be shown. By downloading and using the software, you agree to fully comply with the terms and conditions of the [NVIDIA Software License Agreement](#).

Operating System	Linux	Windows				
Architecture	x86_64	arm64-sbsa	aarch64-jetson			
Distribution	Tarball	Debian	OpenSUSE	RHEL	Rocky	Ubuntu
Version	20.04	22.04	24.04			
Installer Type	deb (local)	deb (network)				

Download Installer for Linux Ubuntu 24.04 x86_64

The base installer is available for download below.

> Base Installer

Download (1.2 GB)

Installation Instructions:

```
$ wget https://developer.download.nvidia.com/compute/cudnn/9.6.0/local_installers/cudnn-local-repo-ubuntu2404-9.6.0_1.0-1_amd64.deb
$ sudo dpkg -i cudnn-local-repo-ubuntu2404-9.6.0_1.0-1_amd64.deb
$ sudo cp /var/cudnn-local-repo-ubuntu2404-9.6.0/cudnn-*keyring.gpg /usr/share/keyrings/
$ sudo apt-get update
$ sudo apt-get -y install cudnn
```

To install for CUDA 11, perform the above configuration but install the CUDA 11 specific package:

```
$ sudo apt-get -y install cudnn-cuda-11
```

To install for CUDA 12, perform the above configuration but install the CUDA 12 specific package:

```
$ sudo apt-get -y install cudnn-cuda-12
```

Installation commands for cuDNN installation

```
wget https://developer.download.nvidia.com/compute/cudnn/9.6.0/local_installers/
sudo dpkg -i cudnn-local-repo-ubuntu2404-9.6.0_1.0-1_amd64.deb
sudo cp /var/cudnn-local-repo-ubuntu2404-9.6.0/cudnn-*keyring.gpg /usr/share/ke
sudo apt-get update
sudo apt-get -y install cudnn-cuda-12
```

This installed cuDNN libraries linked specifically against CUDA 12, ensuring compatibility with our toolkit and ML frameworks.

Optional: Install and Test cuDNN Samples

To test the installation and explore sample projects, we also installed cuDNN's sample code:

```
sudo apt-get -y install libcudnn9-samples  
cp -r cudnn_samples_v9/ ~/source/cudnn_samples_v9  
cd ~/source/cudnn_samples_v9
```

Note: These commands were run and tested on April 24, 2025, inside Ubuntu 24.04 on WSL2. All sample code compiled successfully without errors.

Step 4: Installing PyTorch 2.7 with CUDA 12.6

With CUDA and cuDNN installed and working, we moved on to installing **PyTorch 2.7** — the most recent stable release that supports CUDA 12.6, as of April 2025.

Get Momin Aman's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

We started by heading over to the official [PyTorch installation page](#) and selecting the right configuration for our system:

PyTorch Build	Stable (2.6.0)		Preview (Nightly)	
Your OS	Linux	Mac	Windows	
Package	Conda	Pip	LibTorch	Source
Language	Python		C++ / Java	
Compute Platform	CUDA 11.8	CUDA 12.4	CUDA 12.6	ROCm 6.2.4
Run this Command:	<pre>pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu126</pre>			

Before running any installation commands, we double-checked version support using the PyTorch RELEASE.md compatibility matrix from [here](#). This helped ensure that PyTorch 2.7 and CUDA 12.6 would work smoothly together, and which cuDNN version PyTorch expected internally.

Release Compatibility Matrix					
Following is the Release Compatibility Matrix for PyTorch releases:					
PyTorch version	Python	C++	Stable CUDA	Experimental CUDA	Stable ROCm
2.7	>=3.9, <=3.13, (3.13t experimental)	C++17	CUDA 11.8 (CUDNN 9.1.0.70), CUDA 12.6 (CUDNN 9.5.1.17)	CUDA 12.8 (CUDNN 9.7.1.26)	ROCm 6.3
2.6	>=3.9, <=3.13, (3.13t experimental)	C++17	CUDA 11.8, CUDA 12.4 (CUDNN 9.1.0.70)	CUDA 12.6 (CUDNN 9.5.1.17)	ROCm 6.2.4
2.5	>=3.9, <=3.12, (3.13 experimental)	C++17	CUDA 11.8, CUDA 12.1, CUDA 12.4, CUDNN 9.1.0.70	None	ROCm 6.2
2.4	>=3.8, <=3.12	C++17	CUDA 11.8, CUDA 12.1, CUDNN 9.1.0.70	CUDA 12.4, CUDNN 9.1.0.70	ROCm 6.1
2.3	>=3.8, <=3.11, (3.12 experimental)	C++17	CUDA 11.8, CUDNN 8.7.0.84	CUDA 12.1, CUDNN 8.9.2.26	ROCm 6.0

Minimum CUDA Toolkit and cuDNN versions needed for a specific PyTorch version

Setting Up the Conda Environment

We used Miniconda to manage isolated environments. Here's how we created the environment and installed PyTorch:

```
# Optional: Install ipykernel in your base conda env first
pip install ipykernel

# Create a new environment with Python 3.11
conda create -n torch27 python=3.11
conda activate torch27

# Install PyTorch 2.7 with CUDA 12.6
pip3 install torch torchvision torchaudio --index-url https://download.pytorch.o
```

Verifying PyTorch Installation

Once installed, we opened Python inside the environment and ran:

```
import torch
print(torch.cuda.is_available())
```

If the output is True, it means your PyTorch setup is successfully using the GPU via CUDA.

Why We Created Separate Environments for PyTorch

Before diving into the TensorFlow installation, it's important to explain why we use different Conda environments for each ML framework: PyTorch and TensorFlow.

During our setup, we initially tried installing both frameworks in the same environment (ml) to save time and manageability. But very quickly, version conflicts began to surface:

- PyTorch 2.7 requires cuDNN 9.5 internally.

- TensorFlow 2.13 (planned) depended on cuDNN 9.3 and expected slightly different CUDA bindings.
- Even the Python and dependency versions overlapped or clashed (e.g., one needed ninja, the other didn't).

These incompatibilities made it clear that trying to maintain both frameworks in a single environment could lead to frustrating, hard-to-debug issues, especially when testing GPU availability and runtime behavior.

Lesson Learned: Isolate Environments

So instead, we followed a well-known best practice:

Use isolated environments per framework:

This ensures stable setups and easier upgrades/downgrades per framework without cross-contamination of dependencies.

We created a clean Conda environment for PyTorch:

```
conda create -n torch27 python=3.11
conda activate torch27
```

We'll soon repeat the same for TensorFlow (in a separate env like tf218), keeping installations clean, versioned, and easy to manage.

This also gives flexibility to try out experimental builds or downgrade specific packages without worrying about one framework breaking the other.

Step 5: Installing TensorFlow 2.18 with GPU Support

With PyTorch working perfectly in its own environment, we moved on to setting up **TensorFlow 2.18** in its own dedicated Conda environment to avoid version clashes and package conflicts.

Note: We're keeping TensorFlow separate because PyTorch and TensorFlow often require different cuDNN versions, build flags, and dependencies. Installing both in the same environment is risky and can break one or both frameworks.

Creating the TensorFlow Environment

We created a clean Conda environment using Python 3.11:

```
conda create -n tf218 python=3.11  
conda activate tf218
```

Then installed TensorFlow 2.18 with GPU support:

```
python3 -m pip install tensorflow[and-cuda]==2.18
```

The [and-cuda] tag ensures that TensorFlow pulls in the necessary CUDA and cuDNN libraries directly, rather than relying on global system installations.

Verifying TensorFlow GPU Access

Once installed, we tested the setup using this command:

```
python3 -c "import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))"
```

Expected Output:

```
[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```



```
115 conda activate torch27
116 pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu126
117 python3 -c "import torch; torch.cuda.is_available()"
118 python
119 python3 -c "import torch; torch.cuda.is_available()"
120 python3 -c "import torch; torch.cuda.is_available();"
121 python3 -c "import torch; print(torch.cuda.is_available())"
122 conda deactivate
123 conda create -n tf218 python=3.11
124 conda activate tf218
125 history
(tf218) nomln@Momin: $
(tf218) nomln@Momin: $ python3 -c "import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))"
2025-04-23 08:46:25.722564: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2025-04-23 08:46:25.732618: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:477] Unable to register cuFFT factory: Attempting to register factory for plugin cuFFT when one has already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
E0000 00:00:1745423185.742784 2429 cuda_dnn.cc:8310] Unable to register cuDNN factory: Attempting to register factory for plugin cuDNN when one has already been registered
E0000 00:00:1745423185.745497 2429 cuda_blas.cc:1418] Unable to register cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has already been registered
2025-04-23 08:46:25.755761: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
(tf218) nomln@Momin: $
```

This confirmed that TensorFlow was properly linked with CUDA 12.6 and able to use the GPU. Although some warnings about plugin registration and performance optimizations appeared, they're common and non-blocking unless you're compiling TensorFlow from source. These warnings go away when code is run the next time. So you can safely ignore these.

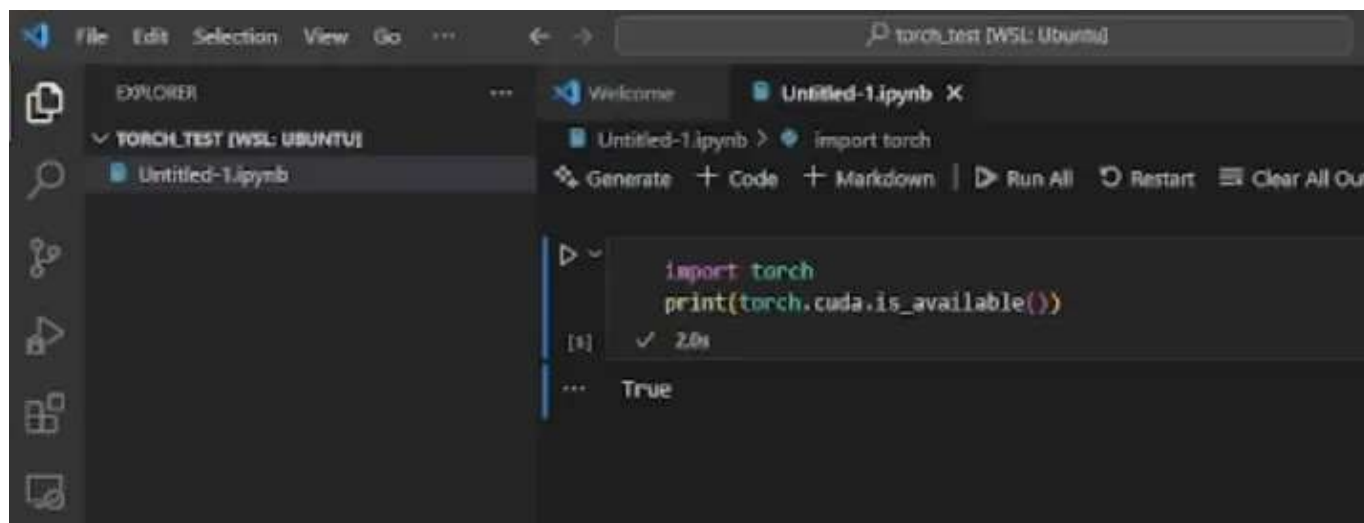
We now had both **PyTorch 2.7** and **TensorFlow 2.18** installed, tested, and GPU-accelerated — each in its own stable sandbox.

Step 6: Running Jupyter Lab

To test the environments visually and run ML code interactively, we set up Jupyter Notebooks with each Conda environment:

```
pip install ipykernel
python -m ipykernel install --user --name=tf218 --display-name "TensorFlow 2.18"
```

We used VS Code's Jupyter plugin to open .ipynb files and select either Torch or TensorFlow kernels depending on what we were working on.



Final Thoughts

Here's what we installed and tested so far:

Cuda Drivers	572.16
Cuda SDK	12.6.3
cuDNN	9.6
Python	3.11
Tensorflow	2.18.0
Pytorch	2.7
TensorRT	10.7

Setting up ML environments with GPU support isn't always straightforward, especially when managing multiple frameworks, SDK versions, and deep learning libraries.

But documenting as we went, co-debugging, cross-referencing NVIDIA support matrices, and verifying everything *step by step* helped us build confidence and clarity. And now, everything is running smoothly, with clean environments, GPU acceleration, and no hidden conflicts.

Whether you're working solo or with a friend (like we did), **having a repeatable, validated dev log is your best defense against future setup pain.**

Happy experimenting, training, and deploying!

Machine Learning

Cuda

Pytorch

TensorFlow

Wsl 2



Written by Momin Aman

9 followers · 20 following

Follow

Immersive Systems Architect & AI Researcher. Engineering the convergence of Intelligence and Space. 138M+ user scale. NYU Alumni. | www.mominaman.com
